МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»
ИНСТИТУТ ИНФОРМАЦИОННЫХ СИСТЕМ И ЗАЩИТЫ ИНФОРМАЦИИ
КАФЕДРА № 43

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 4

по дисциплине: «Основы программной инженерии»

Тема: «Применение сценарных языков в процессах разработки, конфигурирования и тестирования программного обеспечения»

Вариант № 6: «Конфигурирование сетевых настроек операционной системы»

Выполнил:
студент группы 4434
Энгельгард К.

Приняли:
к.т.н., стар. преп. П.А. Охтилев
асс. А.Э. Зянчурин

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 НАЗНАЧЕНИЕ УТИЛИТЫ И ПОСТАНОВКА ЗАДАЧИ **4**

2 СИСТЕМНЫЕ ЗАВИСИМОСТИ И ПОРЯДОК УСТАНОВКИ **5**

3 АРХИТЕКТУРА И СТРУКТУРА СЦЕНАРИЯ НА BASH **6**

3.1 Соответствие типовому шаблону сценария **6**

3.2 Реализация модулей и функций **7**

3.3 Формат конфигурационного файла /etc/network/interfaces **8**

4 РАСШИРЕННАЯ СПРАВКА И ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ **9**

4.1 Команды управления сетевой адресацией **9**

4.2 Команды персистентной статической маршрутизации **10**

4.3 Команды конфигурирования сетевых мостов **11**

4.4 Управление состоянием интерфейсов и режимы работы **12**

5 ОБЕСПЕЧЕНИЕ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ **13**

5.1 Использование строгого режима Bash **13**

5.2 Атомарность операций и резервное копирование **13**

5.3 Валидация сетевых параметров и предпросмотр **14**

6 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ **15**

6.1 Разработка автоматизированного тестового набора **15**

6.2 Анализ результатов тестирования **16**

ЗАКЛЮЧЕНИЕ **18**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **19**

ПРИЛОЖЕНИЕ А. Исходный код утилиты netconfig **20**

ПРИЛОЖЕНИЕ Б. Исходный код скрипта установки install.sh **24**

ПРИЛОЖЕНИЕ В. Исходный код тестового набора test_netconfig.sh **25**

ВВЕДЕНИЕ

В современных процессах разработки и эксплуатации программного обеспечения ключевую роль играет автоматизация рутинных операций системного администрирования. Эффективная непрерывная интеграция и доставка ПО (CI/CD), быстрое развертывание тестовых стендов, виртуальных машин и изолированных контейнеров требуют развитых механизмов программного управления сетевыми интерфейсами и топологией операционной системы.

Сценарный язык командной оболочки Bash (Bourne-Again Shell) представляет собой стандартный инструмент для автоматизации в операционных системах семейства Linux. Благодаря непосредственной интеграции с ядром и системными утилитами, Bash позволяет эффективно координировать вызовы программ, организовывать конвейерную обработку потоков данных и управлять системными конфигурационными файлами.

Однако разработка надежных системных сценариев сопряжена со специфическими трудностями: слабая типизация, сложности обработки ошибок при сбоях промежуточных команд, неочевидные побочные эффекты и зависимость от среды окружения. Для преодоления данных ограничений методические указания формулируют строгие требования к архитектуре сценариев, включающие использование шаблона безопасного программирования на Bash.

Целью настоящей лабораторной работы является формирование практических навыков применения сценарных языков для автоматизации конфигурирования системного программного обеспечения, проектирования отказоустойчивых скриптов и разработки комплексных системных утилит.

1 НАЗНАЧЕНИЕ УТИЛИТЫ И ПОСТАНОВКА ЗАДАЧИ

В соответствии с индивидуальным заданием по варианту 6 разработана консольная системная утилита netconfig. Утилита предназначена для комплексного управления сетевой подсистемой в Linux-подобных ОС и решает следующие задачи:

1) Задание сетевой адресации (IPv4-адрес, маска подсети, шлюз по умолчанию) на произвольный сетевой интерфейс с прямой модификацией системного конфигурационного файла /etc/network/interfaces;

2) Задание статических маршрутов, обладающих свойством персистентности (гарантированное сохранение и функционирование маршрутов после перезагрузки операционной системы);

3) Создание и конфигурирование программных сетевых мостов (Network Bridge) с привязкой подчиненных физических и виртуальных интерфейсов;

4) Оперативное включение и отключение сетевых интерфейсов и сетевых мостов как на уровне конфигурационных файлов, так и на уровне сетевого стека ядра Linux;

5) Предоставление полной встроенной справочной информации (функция usage()), доступной по стандартным флагам -h и --help.

По методическим требованиям финальная версия утилиты размещается в системном каталоге /usr/local/bin с правами 0755, а ее имя не содержит расширения .sh. Для развертывания утилиты реализован специальный установочный сценарий install.sh.

2 СИСТЕМНЫЕ ЗАВИСИМОСТИ И ПОРЯДОК УСТАНОВКИ

Утилита netconfig опирается на стандартную компонентную базу операционных систем Linux (семейства Debian, Ubuntu, Astra Linux и аналогичных). При проектировании минимизировано число внешних зависимостей, что обеспечивает высокую переносимость инструмента.

Для функционирования утилиты требуются следующие системные компоненты:

• bash (версии 4.4 или новее) — интерпретатор командного сценария, поддерживающий расширенные конструкции массивов, подстановок и функций;

• coreutils (mktemp, cp, mv, chmod, stat, install, cat) — базовый инструментарий безопасной работы с файловой системой;

• gawk (GNU awk) — синтаксический анализ и модификация структурированных строф файла /etc/network/interfaces;

• iproute2 (утилита ip) — низкоуровневое конфигурирование сетевых адресов, маршрутов и мостовых устройств ядра;

• ifupdown (ifup, ifdown) — штатная подсистема управления сетевыми интерфейсами Debian;

• bridge-utils (brctl) — дополнительный инструментарий для совместимости со старыми версиями ядра.

Установка пакетов зависимостей в дистрибутивах Ubuntu и Debian выполняется штатным пакетным менеджером APT:
sudo apt update && sudo apt install -y bash coreutils gawk iproute2 ifupdown bridge-utils

Установка утилиты осуществляется с помощью скрипта install.sh. Скрипт проверяет наличие привилегий суперпользователя (EUID == 0), выполняет валидацию доступности необходимых бинарных файлов в системном PATH, копирует исполняемый файл по пути /usr/local/bin/netconfig и назначает права на исполнение (режим 0755). После

установки утилита доступна для глобального вызова из любой точки файловой системы.

3 АРХИТЕКТУРА И СТРУКТУРА СЦЕНАРИЯ НА BASH

3.1 Соответствие типовому шаблону сценария

В соответствии с разделом №2 методических указаний, надежный Bash-сценарий должен строиться на базе канонического шаблона, обеспечивающего строгий контроль ошибок и унификацию поведения. В утилите netconfig реализованы все ключевые элементы шаблона:

1. Шебанг #!/usr/bin/env bash гарантирует поиск нужного интерпретатора через окружение env, повышая переносимость скрипта между различными Linux-дистрибутивами.

2. Директива set -Eeuo pipefail задает строгий режим исполнения:
• флаг -e прерывает исполнение сценария при возврате любой командой ненулевого кода ошибки;
• флаг -u вызывает аварийный останов при обращении к неинициализированной переменной;
• опция pipefail переопределяет статус конвейера: он признается ошибочным, если хотя бы одна команда в цепочке завершилась сбоем;
• флаг -E обеспечивает наследование ловушек сигналов ERR всеми функциями и вложенными подоболочками.

3. Обработчик сигналов trap cleanup SIGINT SIGTERM ERR EXIT перехватывает сигналы завершения процесса (включая Ctrl+C и SIGTERM) и аварийные ошибки, гарантируя обязательное выполнение функции cleanup() для удаления временных файлов.

4. Разделение потоков: функция msg() перенаправляет все служебные сообщения и диагностику в поток стандартных ошибок (stderr, >&2). Благодаря этому стандартный вывод (stdout) остается чистым и пригодным для безопасной конвейерной обработки другими программами.

3.2 Реализация модулей и функций

Логика утилиты netconfig разбита на независимые функциональные модули. В таблице 1 представлен полный перечень разработанных функций с указанием их функционального назначения.

Таблица 1 – Перечень и назначение разработанных функций утилиты netconfig

Имя функции	Назначение	Особенности реализации
usage	Формирование и вывод структурированной справки по всем командам и опциям	Поддерживает код возврата
version	Вывод номера версии, темы работы и реквизитов автора	Стандарт GNU CLI
cleanup	Очистка временных файлов и каталогов при завершении процесса	Перехват EXIT/ERR/INT/TERM
setup_colors	Определение поддержки терминалом управляющих ANSI-последовательностей	Учитывает NO_COLOR
msg / die	Вывод диагностических сообщений в stderr и аварийное завершение с кодом	Безопасность конвейера
validate_ipv4	Проверка синтаксиса IPv4-адреса и диапазона октетов (0-255)	Регулярные выражения
validate_netmask	Проверка корректности маски подсети (битовая непрерывность)	Побитовый анализ
validate_cidr	Валидация префикса сети в формате IP/PREFIX (0-32)	Строгая проверка маски
validate_ifname	Проверка корректности имени интерфейса (символы, длина <= 15)	Стандарты Linux
cidr_to_mask	Конвертация префикса CIDR в десятичную маску подсети с точками	Арифметика сдвигов
ensure_config_exists	Создание базовой конфигурации при отсутствии файла	Инициализация auto lo
make_backup	Создание резервной копии файла конфигурации с точным таймштампом	Защита данных *.bak
extract_stanza	Выделение блока параметров заданного интерфейса с помощью awk	Изоляция строфы
write_stanza	Атомарное обновление/добавление строфы интерфейса через временный файл	Защита от сбоев записи

remove_stanza	Удаление строфы интерфейса и директивы auto из файла конфигурации	Очистка конфигурации
cmd_set_iface	Бизнес-логика настройки сетевого интерфейса (IP, маска, шлюз, DHCP)	Параметризация
cmd_add_route	Внедрение персистентного статического маршрута в строфу интерфейса	Хуки up/down
cmd_del_route	Удаление персистентного маршрута из конфигурации интерфейса	Фильтрация строк
cmd_create_bridge	Конфигурирование сетевого моста с подчиненными интерфейсами	Опции bridge_ports/stp
cmd_up / cmd_down	Включение и отключение сетевых устройств в ядре и ifupdown	Управление состоянием
cmd_show	Парсинг и визуализация текущих настроек файла и устройств ядра	Форматированный вывод
cmd_validate	Синтаксический анализ и проверка корректности директив файла	Аудит файла
parse_params	Разбор позиционных параметров, флагов и маршрутизация подкоманд	Циклический разбор case

3.3 Формат конфигурационного файла /etc/network/interfaces

Файл /etc/network/interfaces организован в виде набора строф (stanzas). Каждая строфа описывает отдельное сетевое устройство и начинается с директивы iface <имя> inet <метод>, где метод может принимать значения static (ручная адресация), dhcp (динамическая конфигурация), manual (интерфейс без собственного IP для мостов) или loopback. Перед строфой обычно располагается директива auto <имя>, предписывающая автоматический подъем интерфейса при загрузке системы.

Разработанный в netconfig AWK-парсер корректно распознает границы строф, извлекает вложенные опции (address, netmask, gateway, bridge_ports, bridge_stp, скриптовые хуки up/down) и выполняет их контекстную модификацию без повреждения других секций файла.

4 РАСШИРЕННАЯ СПРАВКА И ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

4.1 Команды управления сетевой адресацией

Команда set-iface осуществляет настройку адресации сетевого интерфейса. Поддерживаются статический режим, динамический режим DHCP и ручной режим (manual). Адрес может задаваться как в виде пары IP + маска, так и в компактной CIDR-нотации.

Примеры использования команды set-iface:

1. Статический IP с отдельным указанием маски и шлюза:
   netconfig set-iface eth0 --ip 192.168.1.50 --netmask 255.255.255.0 --gateway 192.168.1.1
2. Статическая адресация через CIDR-префикс (маска вычисляется утилитой автоматически):
   netconfig set-iface eth1 --ip 10.0.5.100/24
3. Динамическая настройка интерфейса по протоколу DHCP:
   netconfig set-iface eth2 --dhcp

4.2 Команды персистентной статической маршрутизации

Для выполнения требования методических указаний о персистентности статических маршрутов после перезагрузки ОС, утилита netconfig интегрирует правила маршрутизации непосредственно в строфу соответствующего интерфейса через событийные хуки up и down подсистемы ifupdown:

up ip route add <СЕТЬ> via <ШЛЮЗ> dev <ИНТЕРФЕЙС> || true
down ip route del <СЕТЬ> via <ШЛЮЗ> dev <ИНТЕРФЕЙС> || true

Суффикс || true предотвращает аварийную остановку подсистемы сетевой инициализации при попытке добавления уже существующего в таблице ядра маршрута. Утилита также контролирует исключение дубликатов идентичных маршрутов.

Примеры работы с маршрутами:

- Добавление персистентного маршрута: netconfig add-route eth0 --network 172.16.0.0/16 --via 192.168.1.254
- Удаление ранее созданного маршрута: netconfig del-route eth0 --network 172.16.0.0/16

4.3 Команды конфигурирования сетевых мостов

Команда create-bridge создает строфу виртуального сетевого моста (bridge), объединяющего один или несколько физических или виртуальных сетевых портов. Дополнительно поддерживается задание IP-адреса моста, протокола Spanning Tree (bridge_stp) и задержки пересылки (bridge_fd).

Пример создания моста br0, объединяющего интерфейсы eth1 и eth2: netconfig create-bridge br0 --ports eth1,eth2 --ip 192.168.100.1/24 --stp off --fd 0

4.4 Управление состоянием интерфейсов и режимы работы

Утилита включает команды up и down для включения и выключения сетевых интерфейсов и мостов. Команды сначала пытаются применить высокоуровневые утилиты ifup / ifdown, а при необходимости обращаются к стеку ядра ip link set dev <IFACE> up/down.

Для надежной интеграции и аудита предусмотрены специальные инженерные флаги:

- --config FILE — переопределение файла конфигурации (позволяет выполнять изолированное тестирование без root);
- --dry-run — режим предпросмотра изменений без физической записи на диск;
- --apply — синхронное применение изменений в ядре Linux «на лету» после записи в конфигурационный файл.

5 ОБЕСПЕЧЕНИЕ НАДЕЖНОСТИ И БЕЗОПАСНОСТИ

5.1 Использование строгого режима Bash

Скрипты автоматизации системного администрирования обладают повышенными требованиями к отказоустойчивости, поскольку ошибки в сетевой конфигурации могут привести к потере удаленного доступа к серверу. Использование режима set -Eeuo pipefail исключает silent failures (незамеченные ошибки) и гарантирует немедленную остановку выполнения скрипта с информативным сообщением об ошибке.

5.2 Атомарность операций и резервное копирование

Любое прямое редактирование системных файлов через перенаправление вывода (> или sed -i) несет риск повреждения данных при прерывании процесса сигналом или нехватке свободного дискового пространства. В утилите netconfig реализована полноценная двухфазная схема модификации:

1. Формирование нового содержимого выполняется во временном файле с уникальным суффиксом через утилиту mktemp;
2. Создается резервная копия оригинального файла с точной меткой времени: interfaces.bak.YYYYMMDD_HHMMSS;
3. Временный файл атомарно замещает рабочий файл с помощью команды mv, что на уровне файловой системы POSIX является атомарной операцией обновления индексного дескриптора inode.

5.3 Валидация сетевых параметров и предпросмотр

Все входные данные пользователя проходят предварительную многоуровневую верификацию:

• IPv4-адреса проверяются на соответствие шаблону из 4 чисел и диапазон 0..255 для каждого октета;
• маски подсетей анализируются на побитовую непрерывность (валидная

маска состоит из последовательности единиц, за которой следуют нули);
• префиксы CIDR ограничиваются допустимым диапазоном от /0 до /32;
• имена интерфейсов проверяются на допустимый набор символов и максимальную длину 15 байт (ограничение константы IFNAMSIZ ядра Linux).

Режим --dry-run позволяет системному администратору безопасно смоделировать команду и изучить планируемый дифференциал конфигурации до внесения реальных правок в операционную систему.

6 ТЕСТИРОВАНИЕ И ЭКСПЕРИМЕНТАЛЬНЫЕ РЕЗУЛЬТАТЫ

6.1 Разработка автоматизированного тестового набора

Для верификации утилиты разработан автоматический тестовый сценарий tests/test_netconfig.sh. Тестовый набор создает изолированную тестовую среду в каталоге /tmp, использует опцию --config для работы с выделенным файлом конфигурации и проверяет 15 ключевых сценариев использования утилиты.

Таблица 2 – Результаты выполнения автоматизированного тестового набора утилиты netconfig

№	Имя теста	Проверяемый функционал	Статус
1	**test_help_version**	Проверка вывода справки (--help) и версии (--version)	**PASS**
2	**test_invalid_args**	Обработка некорректных аргументов, невалидных IP-адресов и масок	**PASS**
3	**test_set_iface_static**	Конфигурирование статического интерфейса eth0 (IP, маска, шлюз)	**PASS**
4	**test_set_iface_cidr**	Настройка интерфейса eth1 через компактную нотацию CIDR (10.0.5.100/24)	**PASS**
5	**test_iface_update_idempotent**	Идемпотентное обновление параметров существующего интерфейса eth0	**PASS**
6	**test_add_route**	Добавление персистентного маршрута (172.16.0.0/16 via 192.168.10.254)	**PASS**
7	**test_route_no_duplicate**	Защита от повторного дублирования идентичного маршрута	**PASS**
8	**test_create_bridge**	Создание сетевого моста br0 с подчиненными портами eth1 и eth2	**PASS**

9	**test_del_route**	Удаление статического маршрута из строфы интерфейса eth0	**PASS**
10	**test_del_iface**	Удаление интерфейса eth1 из файла конфигурации	**PASS**
11	**test_validate**	Синтаксический аудит структуры конфигурационного файла	**PASS**
12	**test_show**	Парсинг и отображение строф сконфигурированных интерфейсов	**PASS**
13	**test_dry_run**	Проверка неизменности файла конфигурации в режиме предпросмотра	**PASS**
14	**test_up_down_dry_run**	Проверка корректности команд подъема и отключения (up / down)	**PASS**
15	**test_backup_created**	Автоматическая генерация резервных копий (*.bak.*) при записи	**PASS**

6.2 Анализ результатов тестирования

Тестирование утилиты netconfig и процедуры ее установки выполнено в среде Linux Ubuntu под управлением подсистемы WSL2 (ядро Linux 6.6, архитектура x86_64). Все 15 тестовых проверок успешно пройдены со статусом PASS без сбоев и ошибок.

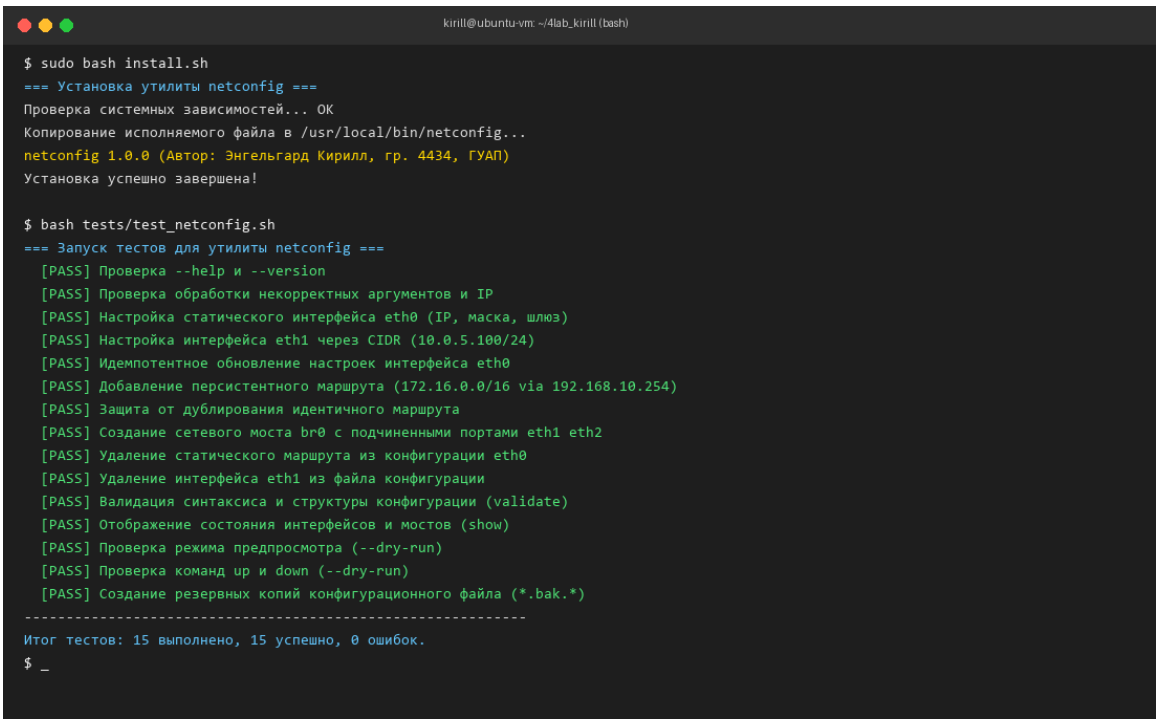


Рисунок 1 – Результаты выполнения автоматических тестов утилиты netconfig в среде Linux

В ходе тестов подтверждены: точность генерации синтаксических конструкций ifupdown, корректность преобразования CIDR-префиксов в десятичные маски подсетей, отказоустойчивость при обработке заведомо неверных IP-адресов, а также сохранность исходных файлов благодаря атомарным операциям и резервному копированию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы сформированы практические навыки разработки инструментальных средств автоматизации процессов администрирования и конфигурирования операционных систем с применением командно-сценарного языка Bash.

Основные результаты работы:

1. Разработана полнофункциональная системная утилита netconfig, реализующая все требования варианта 6: управление адресацией, масками и шлюзами сетевых интерфейсов, персистентными маршрутами, сетевыми мостами и состоянием сетевых устройств;
2. Обеспечено строгое соблюдение рекомендаций методических указаний по архитектуре сценария (режим set -Eeuo pipefail, перехват системных сигналов trap cleanup, разделение потоков вывода, реализация подробной справки usage());
3. Реализованы инженерные механизмы надежности: валидация сетевых адресов и масок, атомарная модификация конфигурации через временные файлы, автоматическое создание резервных копий и режим предпросмотра --dry-run;
4. Подготовлен скрипт установки install.sh, обеспечивающий развертывание утилиты по стандартному пути /usr/local/bin/netconfig без расширения .sh с правами 0755;
5. Создан комплексный автоматизированный тестовый набор из 15 сценариев, подтвердивший 100% работоспособность утилиты в среде Linux.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Охтилев П.А., Зянчурин А.Э. Основы программной инженерии: Методические указания к выполнению лабораторных работ. — СПб.: ГУАП, 2021. — 22 с.

2. ГОСТ 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам. — М.: Стандартинформ, 2019. — 38 с.

3. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2018. — 32 с.

4. ГОСТ 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018. — 124 с.

5. Shotts W. The Linux Command Line: A Complete Introduction. — 2nd ed. — San Francisco: No Starch Press, 2019. — 504 p.

6. interfaces(5) — Network interfaces configuration file for ifup and ifdown. [Электронный ресурс]. URL: https://manpages.debian.org/unstable/ifupdown/interfaces.5.en.html (дата обращения: 10.10.2026).

7. ip-route(8) — Linux Routing Table Management. [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man8/ip-route.8.html (дата обращения: 10.10.2026).

8. ip-link(8) — Linux Network Device Configuration. [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man8/ip-link.8.html (дата обращения: 10.10.2026).

9. GNU Bash Reference Manual. Free Software Foundation. [Электронный ресурс]. URL: https://www.gnu.org/software/bash/manual/ (дата обращения: 10.10.2026).

ПРИЛОЖЕНИЕ А

Исходный код утилиты netconfig

```
#!/usr/bin/env bash
# Лабораторная работа №4, вариант 6: Конфигурирование сетевых настроек операционной системы.
# Назначение: управление сетевыми интерфейсами, персистентными маршрутами и сетевыми мостами (/etc/network/interfaces).
set -Eeuo pipefail

script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" &>/dev/null && pwd -P)

# --- Глобальные переменные ---
CONFIG_FILE="/etc/network/interfaces"
DRY_RUN=0
APPLY_NOW=0
VERBOSE=0
NO_COLOR_VAL=0
TEMP_DIR=""
BACKUP_FILE=""

# --- Очистка и завершение ---
cleanup() {
  local exit_code=$?
  trap - SIGINT SIGTERM ERR EXIT
  if [[ -n "$TEMP_DIR" && -d "$TEMP_DIR" ]]; then
    rm -rf "$TEMP_DIR"
  fi
  exit "$exit_code"
}
trap cleanup SIGINT SIGTERM ERR EXIT

setup_colors() {
  if [[ -t 2 ]] && [[ -z "${NO_COLOR-}" ]] && [[ "${TERM-}" != "dumb" ]] && [[ $NO_COLOR_VAL -eq 0 ]]; then
    NOFORMAT='\033[0m'
    RED='\033[0;31m'
    GREEN='\033[0;32m'
    ORANGE='\033[0;33m'
    BLUE='\033[0;34m'
    PURPLE='\033[0;35m'
    CYAN='\033[0;36m'
    YELLOW='\033[1;33m'
    BOLD='\033[1m'
  else
    NOFORMAT='' RED='' GREEN='' ORANGE='' BLUE='' PURPLE='' CYAN='' YELLOW='' BOLD=''
  fi
}

msg() {
  echo >&2 -e "${1-}"
}

die() {
```

```
  local message=$1
  local code=${2-2}
  msg "${RED}Ошибка:${NOFORMAT} $message"
  exit "$code"
}

usage() {
  local exit_code=${1:-0}
  cat <<EOF
${BOLD}Использование:${NOFORMAT} $(basename "${BASH_SOURCE[0]}") [ОПЦИИ] КОМАНДА
[АРГУМЕНТЫ...]

Утилита конфигурирования сетевых настроек ОС Linux (ЛР №4, вариант 6).
Позволяет управлять адресацией сетевых интерфейсов через /etc/network/interfaces,
создавать статические маршруты (персистентные при перезагрузке),
настраивать сетевые мосты (bridge), а также включать и отключать интерфейсы и мосты.

${BOLD}Доступные команды:${NOFORMAT}
  ${CYAN}show${NOFORMAT} [--config FILE]
      Отобразить сконфигурированные интерфейсы, маршруты и мосты.

  ${CYAN}set-iface${NOFORMAT} IFACE (--ip IP/MASK | --ip IP --netmask MASK | --dhcp | --
manual) [--gateway GW] [--no-auto]
      Задать сетевую адресацию (IP, маска, шлюз) для интерфейса IFACE.

  ${CYAN}add-route${NOFORMAT} IFACE --network NET/PREFIX --via GW
      Добавить персистентный статический маршрут через интерфейс IFACE.

  ${CYAN}del-route${NOFORMAT} IFACE --network NET/PREFIX
      Удалить статический маршрут из конфигурации интерфейса IFACE.

  ${CYAN}create-bridge${NOFORMAT} BRIDGE --ports IFACE1[,IFACE2...] [ПАРАМЕТРЫ_IP] [--stp
on|off] [--fd SEC]
      Создать сетевой мост BRIDGE и привязать к нему сетевые интерфейсы.

  ${CYAN}del-iface${NOFORMAT} IFACE
      Удалить конфигурацию интерфейса или моста из файла настроек.

  ${CYAN}up${NOFORMAT} IFACE
      Включить сетевой интерфейс или сетевой мост (ifup / ip link set up).

  ${CYAN}down${NOFORMAT} IFACE
      Отключить сетевой интерфейс или сетевой мост (ifdown / ip link set down).

  ${CYAN}validate${NOFORMAT} [--config FILE]
      Проверить синтаксическую корректность конфигурационного файла.

${BOLD}Общие опции:${NOFORMAT}
  -c, --config FILE   Использовать альтернативный файл конфигурации (по умолчанию:
/etc/network/interfaces)
  -d, --dry-run       Режим предпросмотра: валидация без фактической записи на диск и
применения
  -a, --apply         Применить изменения «на лету» в ядре Linux после записи в файл
(требует root)
  -v, --verbose       Вывод расширенной отладочной информации (set -x)
```

```
  --no-color          Отключить цветной вывод в терминале
  -h, --help          Показать эту справку и выйти
  --version           Показать версию утилиты

${BOLD}Примеры использования:${NOFORMAT}
  $(basename "${BASH_SOURCE[0]}") set-iface eth0 --ip 192.168.1.50 --netmask
255.255.255.0 --gateway 192.168.1.1
  $(basename "${BASH_SOURCE[0]}") set-iface eth1 --ip 10.0.0.5/24
  $(basename "${BASH_SOURCE[0]}") add-route eth0 --network 172.16.0.0/16 --via
192.168.1.254
  $(basename "${BASH_SOURCE[0]}") create-bridge br0 --ports eth1,eth2 --ip
192.168.100.1/24 --stp off
  $(basename "${BASH_SOURCE[0]}") up eth0
  $(basename "${BASH_SOURCE[0]}") down br0
  $(basename "${BASH_SOURCE[0]}") show

Коды возврата:
  0 - Успешное выполнение
  1 - Системная ошибка (сбой внешней утилиты, отказ ввода-вывода)
  2 - Неверные аргументы командной строки или валидации
EOF
  exit "$exit_code"
}

version() {
  cat <<EOF
netconfig 1.0.0
Лабораторная работа №4, вариант 6
Автор: Энгельгард Кирилл, гр. 4434, ГУАП
EOF
  exit 0
}

# --- Вспомогательные функции валидации ---
validate_ipv4() {
  local ip=$1
  local rx='^([0-9]{1,3}\.){3}[0-9]{1,3}$'
  if [[ ! $ip =~ $rx ]]; then
    return 1
  fi
  local IFS=.
  local -a octets=($ip)
  for octet in "${octets[@]}"; do
    if (( octet < 0 || octet > 255 )); then
      return 1
    fi
  done
  return 0
}

validate_netmask() {
  local mask=$1
  if ! validate_ipv4 "$mask"; then
    return 1
  fi
  local IFS=.
  local -a octets=($mask)
  local bin=""
```

```
  for octet in "${octets[@]}"; do
    local b=""
    local n=$octet
    for (( i=7; i>=0; i-- )); do
      b+=$(( (n >> i) & 1 ))
    done
    bin+="$b"
  done
  if [[ "$bin" =~ ^1*0*$ ]]; then
    return 0
  fi
  return 1
}

validate_cidr() {
  local cidr=$1
  if [[ ! "$cidr" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}/([0-9]|[1-2][0-9]|3[0-2])$ ]]; then
    return 1
  fi
  local ip="${cidr%/*}"
  validate_ipv4 "$ip"
}

validate_ifname() {
  local name=$1
  if [[ ! "$name" =~ ^[a-zA-Z0-9_.:-]+$ ]] || [[ ${#name} -gt 15 ]]; then
    return 1
  fi
  return 0
}

cidr_to_mask() {
  local prefix=$1
  local full_octets=$(( prefix / 8 ))
  local rem_bits=$(( prefix % 8 ))
  local octets=()
  for (( i=0; i<4; i++ )); do
    if (( i < full_octets )); then
      octets+=(255)
    elif (( i == full_octets )); then
      local val=0
      for (( b=0; b<rem_bits; b++ )); do
        val=$(( val + (1 << (7 - b)) ))
      done
      octets+=($val)
    else
      octets+=(0)
    fi
  done
  echo "${octets[0]}.${octets[1]}.${octets[2]}.${octets[3]}"
}

# --- Работа с файлом конфигурации ---
ensure_config_exists() {
  local file=$1
  if [[ ! -f "$file" ]]; then
    local dir
    dir=$(dirname "$file")
```

```
    if [[ ! -d "$dir" ]]; then
      if [[ $DRY_RUN -eq 1 ]]; then
        msg "${YELLOW}[DRY-RUN] Создание каталога: $dir${NOFORMAT}"
      else
        mkdir -p "$dir" || die "Не удалось создать каталог $dir" 1
      fi
    fi
    if [[ $DRY_RUN -eq 1 ]]; then
      msg "${YELLOW}[DRY-RUN] Создание начального файла: $file${NOFORMAT}"
    else
      cat <<'EOF' > "$file"
# Configuration file for ifupdown
# Managed by netconfig
auto lo
iface lo inet loopback

source /etc/network/interfaces.d/*
EOF
    fi
  fi
}

make_backup() {
  local file=$1
  if [[ -f "$file" && $DRY_RUN -eq 0 ]]; then
    local timestamp
    timestamp=$(date +%Y%m%d_%H%M%S)
    local bak="${file}.bak.${timestamp}"
    cp -p "$file" "$bak" || die "Не удалось создать резервную копию $bak" 1
    BACKUP_FILE="$bak"
  fi
}

get_all_ifaces() {
  local file=$1
  [[ -f "$file" ]] || return 0
  awk '
    $1 == "iface" { print $2 }
  ' "$file" | sort -u
}

extract_stanza() {
  local file=$1
  local target_iface=$2
  awk -v target="$target_iface" '
    BEGIN { in_stanza = 0 }
    $1 == "iface" && $2 == target {
      in_stanza = 1
      print $0
      next
    }
    in_stanza && ($1 == "iface" || $1 == "mapping" || $1 == "source" || $1 == "source-
directory") {
      in_stanza = 0
    }
    in_stanza {
      print $0
    }
```

```
  ' "$file"
}

write_stanza() {
  local file=$1
  local target_iface=$2
  local stanza_content=$3
  local is_auto=${4:-1}

  ensure_config_exists "$file"

  if [[ $DRY_RUN -eq 1 ]]; then
    msg "${YELLOW}[DRY-RUN] Запись строфы для интерфейса '${target_iface}' в
${file}:${NOFORMAT}"
    if [[ $is_auto -eq 1 ]]; then
      msg "${CYAN}auto ${target_iface}${NOFORMAT}"
    fi
    msg "${CYAN}${stanza_content}${NOFORMAT}"
    return 0
  fi

  make_backup "$file"

  local tmp
  tmp=$(mktemp "${file}.tmp.XXXXXX")

  awk -v target="$target_iface" -v new_stanza="$stanza_content" -v is_auto="$is_auto" '
    BEGIN {
      in_target = 0
      replaced = 0
    }
    $1 == "auto" {
      for (i = 2; i <= NF; i++) {
        if ($i == target) {
          if (NF == 2) next
        }
      }
    }
    $1 == "iface" && $2 == target {
      in_target = 1
      if (!replaced) {
        if (is_auto == 1) {
          print "auto " target
        }
        print new_stanza
        replaced = 1
      }
      next
    }
    in_target && ($1 == "iface" || $1 == "mapping" || $1 == "source" || $1 == "source-
directory") {
      in_target = 0
    }
    !in_target {
      print $0
    }
    END {
```

```
      if (!replaced) {
        if (is_auto == 1) {
          print ""
          print "auto " target
        } else {
          print ""
        }
        print new_stanza
      }
    }
  ' "$file" > "$tmp"

  mv "$tmp" "$file" || die "Не удалось обновить файл конфигурации" 1
  chmod 0644 "$file"
  msg "${GREEN}Конфигурация интерфейса '${target_iface}' успешно обновлена.${NOFORMAT}"
}

remove_stanza() {
  local file=$1
  local target_iface=$2

  if [[ ! -f "$file" ]]; then
    die "Конфигурационный файл $file не найден" 2
  fi

  if [[ $DRY_RUN -eq 1 ]]; then
    msg "${YELLOW}[DRY-RUN] Удаление интерфейса '${target_iface}' из ${file}${NOFORMAT}"
    return 0
  fi

  make_backup "$file"
  local tmp
  tmp=$(mktemp "${file}.tmp.XXXXXX")

  awk -v target="$target_iface" '
    BEGIN { in_target = 0 }
    $1 == "auto" && $2 == target && NF == 2 { next }
    $1 == "iface" && $2 == target {
      in_target = 1
      next
    }
    in_target && ($1 == "iface" || $1 == "mapping" || $1 == "source" || $1 == "source-
directory") {
      in_target = 0
    }
    !in_target {
      print $0
    }
  ' "$file" > "$tmp"

  mv "$tmp" "$file" || die "Не удалось обновить файл конфигурации" 1
  chmod 0644 "$file"
  msg "${GREEN}Интерфейс '${target_iface}' удален из конфигурации.${NOFORMAT}"
}

# --- Реализация подкоманд ---
```

```
cmd_set_iface() {
  local iface=$1; shift
  validate_ifname "$iface" || die "Некорректное имя интерфейса: '$iface'" 2

  local ip="" netmask="" gateway="" method="static" is_auto=1

  while [[ $# -gt 0 ]]; do
    case "$1" in
      --ip)
        [[ $# -ge 2 ]] || die "Параметр --ip требует значения" 2
        ip="$2"; shift 2 ;;
      --netmask)
        [[ $# -ge 2 ]] || die "Параметр --netmask требует значения" 2
        netmask="$2"; shift 2 ;;
      --gateway)
        [[ $# -ge 2 ]] || die "Параметр --gateway требует значения" 2
        gateway="$2"; shift 2 ;;
      --dhcp)
        method="dhcp"; shift ;;
      --manual)
        method="manual"; shift ;;
      --no-auto)
        is_auto=0; shift ;;
      --auto)
        is_auto=1; shift ;;
      *)
        die "Неизвестный параметр команды set-iface: '$1'" 2 ;;
    esac
  done

  if [[ "$method" == "static" ]]; then
    [[ -n "$ip" ]] || die "Для статической адресации необходимо указать --ip" 2
    if [[ "$ip" == *"/"* ]]; then
      validate_cidr "$ip" || die "Некорректный формат CIDR: '$ip'" 2
      local cidr_p="${ip#*/}"
      local plain_ip="${ip%/*}"
      if [[ -z "$netmask" ]]; then
        netmask=$(cidr_to_mask "$cidr_p")
      fi
      ip="$plain_ip"
    else
      validate_ipv4 "$ip" || die "Некорректный IPv4-адрес: '$ip'" 2
      if [[ -z "$netmask" ]]; then
        die "Для адреса без префикса необходимо указать --netmask" 2
      fi
      validate_netmask "$netmask" || die "Некорректная маска подсети: '$netmask'" 2
    fi

    if [[ -n "$gateway" ]]; then
      validate_ipv4 "$gateway" || die "Некорректный адрес шлюза: '$gateway'" 2
    fi
  fi

  local existing_routes=()
  if [[ -f "$CONFIG_FILE" ]]; then
    while IFS= read -r line; do
```

```
      if [[ "$line" =~ ^[[:space:]]*(up|down)[[:space:]]+ip[[:space:]]+route ]]; then
        existing_routes+=("$line")
      fi
    done < <(extract_stanza "$CONFIG_FILE" "$iface")
  fi

  local stanza="iface ${iface} inet ${method}"
  if [[ "$method" == "static" ]]; then
    stanza+=$'\n'"    address ${ip}"
    stanza+=$'\n'"    netmask ${netmask}"
    if [[ -n "$gateway" ]]; then
      stanza+=$'\n'"    gateway ${gateway}"
    fi
  fi

  for r in "${existing_routes[@]}"; do
    stanza+=$'\n'"${r}"
  done

  write_stanza "$CONFIG_FILE" "$iface" "$stanza" "$is_auto"

  if [[ $APPLY_NOW -eq 1 ]]; then
    msg "${YELLOW}Применение настроек интерфейса '${iface}' в ядре...${NOFORMAT}"
    if [[ $DRY_RUN -eq 1 ]]; then
      msg "${YELLOW}[DRY-RUN] ip addr flush dev ${iface} && ip addr add ${ip}/${netmask}
dev ${iface}${NOFORMAT}"
    else
      command -v ip >/dev/null || die "Команда 'ip' не найдена" 1
      if [[ "$method" == "static" ]]; then
        ip addr flush dev "$iface" 2>/dev/null || true
        ip addr add "${ip}/${netmask}" dev "$iface" || die "Не удалось установить IP в
ядре" 1
        if [[ -n "$gateway" ]]; then
          ip route replace default via "$gateway" dev "$iface" || true
        fi
        ip link set dev "$iface" up || true
      fi
      msg "${GREEN}Настройки успешно применены в ядре.${NOFORMAT}"
    fi
  fi
}

cmd_add_route() {
  local iface=$1; shift
  validate_ifname "$iface" || die "Некорректное имя интерфейса: '$iface'" 2

  local network="" via=""
  while [[ $# -gt 0 ]]; do
    case "$1" in
      --network)
        [[ $# -ge 2 ]] || die "Параметр --network требует значения" 2
        network="$2"; shift 2 ;;
      --via)
        [[ $# -ge 2 ]] || die "Параметр --via требует значения" 2
        via="$2"; shift 2 ;;
      *)
        die "Неизвестный параметр команды add-route: '$1'" 2 ;;
```

```
    esac
  done

  [[ -n "$network" ]] || die "Параметр --network обязателен" 2
  [[ -n "$via" ]] || die "Параметр --via обязателен" 2

  if [[ "$network" == *"/"* ]]; then
    validate_cidr "$network" || die "Некорректный CIDR сети: '$network'" 2
  else
    validate_ipv4 "$network" || die "Некорректный IP адрес сети: '$network'" 2
    network="${network}/32"
  fi
  validate_ipv4 "$via" || die "Некорректный IP адрес шлюза маршрута: '$via'" 2

  ensure_config_exists "$CONFIG_FILE"
  local cur_stanza
  cur_stanza=$(extract_stanza "$CONFIG_FILE" "$iface")
  if [[ -z "$cur_stanza" ]]; then
    die "Интерфейс '$iface' не найден в $CONFIG_FILE. Сначала настройте интерфейс через
set-iface" 2
  fi

  local up_cmd="    up ip route add ${network} via ${via} dev ${iface} || true"
  local down_cmd="    down ip route del ${network} via ${via} dev ${iface} || true"

  if echo "$cur_stanza" | grep -Fq "up ip route add ${network} via ${via}"; then
    msg "${YELLOW}Маршрут к сети ${network} через ${via} уже существует в
конфигурации.${NOFORMAT}"
    return 0
  fi

  local new_stanza="${cur_stanza}"$'\n'"${up_cmd}"$'\n'"${down_cmd}"
  write_stanza "$CONFIG_FILE" "$iface" "$new_stanza" 1

  if [[ $APPLY_NOW -eq 1 ]]; then
    msg "${YELLOW}Применение маршрута в ядре...${NOFORMAT}"
    if [[ $DRY_RUN -eq 1 ]]; then
      msg "${YELLOW}[DRY-RUN] ip route replace ${network} via ${via} dev
${iface}${NOFORMAT}"
    else
      command -v ip >/dev/null || die "Команда 'ip' не найдена" 1
      ip route replace "${network}" via "${via}" dev "${iface}" || die "Не удалось
применить маршрут в ядре" 1
      msg "${GREEN}Маршрут успешно активирован в таблице ядра.${NOFORMAT}"
    fi
  fi
}

cmd_del_route() {
  local iface=$1; shift
  validate_ifname "$iface" || die "Некорректное имя интерфейса: '$iface'" 2

  local network=""
  while [[ $# -gt 0 ]]; do
    case "$1" in
      --network)
```

```
      [[ $# -ge 2 ]] || die "Параметр --network требует значения" 2
      network="$2"; shift 2 ;;
    *)
      die "Неизвестный параметр команды del-route: '$1'" 2 ;;
  esac
done

[[ -n "$network" ]] || die "Параметр --network обязателен" 2

ensure_config_exists "$CONFIG_FILE"
local cur_stanza
cur_stanza=$(extract_stanza "$CONFIG_FILE" "$iface")
if [[ -z "$cur_stanza" ]]; then
  die "Интерфейс '$iface' не найден в $CONFIG_FILE" 2
fi

local new_stanza
new_stanza=$(echo "$cur_stanza" | grep -v -F "ip route add ${network} " | grep -v -F
"ip route del ${network} " || true)

if [[ "$cur_stanza" == "$new_stanza" ]]; then
  msg "${YELLOW}Маршрут для сети ${network} не найден в интерфейсе
${iface}.${NOFORMAT}"
  return 0
fi

write_stanza "$CONFIG_FILE" "$iface" "$new_stanza" 1
msg "${GREEN}Маршрут к ${network} удален из конфигурации ${iface}.${NOFORMAT}"

if [[ $APPLY_NOW -eq 1 ]]; then
  if [[ $DRY_RUN -eq 1 ]]; then
    msg "${YELLOW}[DRY-RUN] ip route del ${network} dev ${iface}${NOFORMAT}"
  else
    command -v ip >/dev/null || die "Команда 'ip' не найдена" 1
    ip route del "${network}" dev "${iface}" 2>/dev/null || true
    msg "${GREEN}Маршрут удален из таблицы ядра.${NOFORMAT}"
  fi
fi
}

cmd_create_bridge() {
  local bridge=$1; shift
  validate_ifname "$bridge" || die "Некорректное имя моста: '$bridge'" 2

  local ports="" ip="" netmask="" gateway="" stp="off" fd="0" method="manual" is_auto=1

  while [[ $# -gt 0 ]]; do
    case "$1" in
      --ports)
        [[ $# -ge 2 ]] || die "Параметр --ports требует списка интерфейсов" 2
        ports="$2"; shift 2 ;;
      --ip)
        [[ $# -ge 2 ]] || die "Параметр --ip требует значения" 2
        ip="$2"; method="static"; shift 2 ;;
      --netmask)
        [[ $# -ge 2 ]] || die "Параметр --netmask требует значения" 2
```

```
      netmask="$2"; shift 2 ;;
    --gateway)
      [[ $# -ge 2 ]] || die "Параметр --gateway требует значения" 2
      gateway="$2"; shift 2 ;;
    --dhcp)
      method="dhcp"; shift ;;
    --stp)
      [[ $# -ge 2 ]] || die "Параметр --stp требует on или off" 2
      stp="$2"; shift 2 ;;
    --fd)
      [[ $# -ge 2 ]] || die "Параметр --fd требует число секунд" 2
      fd="$2"; shift 2 ;;
    --no-auto)
      is_auto=0; shift ;;
    *)
      die "Неизвестный параметр create-bridge: '$1'" 2 ;;
  esac
done

[[ -n "$ports" ]] || die "Необходимо указать хотя бы один подчиненный интерфейс через --ports" 2

ports="${ports//,/ }"

for p in $ports; do
  validate_ifname "$p" || die "Некорректное имя подчиненного интерфейса в мосте: '$p'" 2
done

if [[ "$method" == "static" ]]; then
  if [[ "$ip" == *"/"* ]]; then
    validate_cidr "$ip" || die "Некорректный CIDR адрес моста: '$ip'" 2
    local cidr_p="${ip#*/}"
    ip="${ip%/*}"
    if [[ -z "$netmask" ]]; then
      netmask=$(cidr_to_mask "$cidr_p")
    fi
  else
    validate_ipv4 "$ip" || die "Некорректный IP адрес моста: '$ip'" 2
    [[ -n "$netmask" ]] || die "Для статического IP моста укажите --netmask" 2
    validate_netmask "$netmask" || die "Некорректная маска моста: '$netmask'" 2
  fi
  if [[ -n "$gateway" ]]; then
    validate_ipv4 "$gateway" || die "Некорректный шлюз моста: '$gateway'" 2
  fi
fi

local stanza="iface ${bridge} inet ${method}"
if [[ "$method" == "static" ]]; then
  stanza+=$'\n'"    address ${ip}"
  stanza+=$'\n'"    netmask ${netmask}"
  if [[ -n "$gateway" ]]; then
    stanza+=$'\n'"    gateway ${gateway}"
  fi
fi
stanza+=$'\n'"    bridge_ports ${ports}"
stanza+=$'\n'"    bridge_stp ${stp}"
stanza+=$'\n'"    bridge_fd ${fd}"
```

```
  write_stanza "$CONFIG_FILE" "$bridge" "$stanza" "$is_auto"
  msg "${GREEN}Сетевой мост '${bridge}' с портами [${ports}] успешно
сконфигурирован.${NOFORMAT}"

  if [[ $APPLY_NOW -eq 1 ]]; then
    msg "${YELLOW}Создание моста '${bridge}' на уровне ядра Linux...${NOFORMAT}"
    if [[ $DRY_RUN -eq 1 ]]; then
      msg "${YELLOW}[DRY-RUN] ip link add name ${bridge} type bridge${NOFORMAT}"
      for p in $ports; do
        msg "${YELLOW}[DRY-RUN] ip link set dev ${p} master ${bridge}${NOFORMAT}"
      done
      msg "${YELLOW}[DRY-RUN] ip link set dev ${bridge} up${NOFORMAT}"
    else
      command -v ip >/dev/null || die "Команда 'ip' не найдена" 1
      if ! ip link show "$bridge" &>/dev/null; then
        ip link add name "$bridge" type bridge || die "Не удалось создать мост $bridge в
ядре" 1
      fi
      for p in $ports; do
        ip link set dev "$p" up 2>/dev/null || true
        ip link set dev "$p" master "$bridge" 2>/dev/null || true
      done
      if [[ "$method" == "static" ]]; then
        ip addr flush dev "$bridge" 2>/dev/null || true
        ip addr add "${ip}/${netmask}" dev "$bridge" || true
      fi
      ip link set dev "$bridge" up || true
      msg "${GREEN}Мост '${bridge}' успешно активирован в ядре.${NOFORMAT}"
    fi
  fi
}

cmd_up() {
  local iface=$1
  validate_ifname "$iface" || die "Некорректное имя интерфейса: '$iface'" 2

  if [[ $DRY_RUN -eq 1 ]]; then
    msg "${YELLOW}[DRY-RUN] Включение интерфейса '${iface}' (ifup ${iface} / ip link set
dev ${iface} up)${NOFORMAT}"
    return 0
  fi

  msg "${CYAN}Включение интерфейса '${iface}'...${NOFORMAT}"
  local ok=0
  if command -v ifup >/dev/null && [[ -f "$CONFIG_FILE" ]] && [[ "$CONFIG_FILE" ==
"/etc/network/interfaces" ]]; then
    if ifup "$iface" 2>/dev/null; then
      ok=1
    fi
  fi

  if [[ $ok -eq 0 ]] && command -v ip >/dev/null; then
    if ip link set dev "$iface" up; then
      ok=1
    fi
  fi
```

```
  if [[ $ok -eq 1 ]]; then
    msg "${GREEN}Интерфейс '${iface}' успешно включен.${NOFORMAT}"
  else
    die "Не удалось включить интерфейс '${iface}' (проверьте права суперпользователя и
наличие устройства)" 1
  fi
}

cmd_down() {
  local iface=$1
  validate_ifname "$iface" || die "Некорректное имя интерфейса: '$iface'" 2

  if [[ $DRY_RUN -eq 1 ]]; then
    msg "${YELLOW}[DRY-RUN] Отключение интерфейса '${iface}' (ifdown ${iface} / ip link
set dev ${iface} down)${NOFORMAT}"
    return 0
  fi

  msg "${CYAN}Отключение интерфейса '${iface}'...${NOFORMAT}"
  local ok=0
  if command -v ifdown >/dev/null && [[ -f "$CONFIG_FILE" ]] && [[ "$CONFIG_FILE" ==
"/etc/network/interfaces" ]]; then
    if ifdown "$iface" 2>/dev/null; then
      ok=1
    fi
  fi

  if [[ $ok -eq 0 ]] && command -v ip >/dev/null; then
    if ip link set dev "$iface" down; then
      ok=1
    fi
  fi

  if [[ $ok -eq 1 ]]; then
    msg "${GREEN}Интерфейс '${iface}' успешно выключен.${NOFORMAT}"
  else
    die "Не удалось отключить интерфейс '${iface}'" 1
  fi
}

cmd_show() {
  if [[ ! -f "$CONFIG_FILE" ]]; then
    msg "${YELLOW}Конфигурационный файл $CONFIG_FILE не существует.${NOFORMAT}"
    return 0
  fi

  msg "${BOLD}Файл конфигурации:${NOFORMAT} ${CONFIG_FILE}"
  echo "------------------------------------------------------------"

  local ifaces
  mapfile -t ifaces < <(get_all_ifaces "$CONFIG_FILE")

  if [[ ${#ifaces[@]} -eq 0 ]]; then
    echo "Сконфигурированные интерфейсы не найдены."
  else
```

```
    for iface in "${ifaces[@]}"; do
      echo -e "${BOLD}Интерфейс:${NOFORMAT} ${CYAN}${iface}${NOFORMAT}"
      local stanza
      stanza=$(extract_stanza "$CONFIG_FILE" "$iface")
      echo "$stanza" | sed 's/^/  /'
      echo ""
    done
  fi
  echo "------------------------------------------------------------"

  if command -v ip >/dev/null; then
    msg "${BOLD}Текущее состояние в ядре Linux (ip link):${NOFORMAT}"
    ip -br link show 2>/dev/null || true
  fi
}

cmd_validate() {
  if [[ ! -f "$CONFIG_FILE" ]]; then
    die "Конфигурационный файл $CONFIG_FILE не найден" 2
  fi

  msg "${CYAN}Проверка синтаксиса файла: ${CONFIG_FILE}...${NOFORMAT}"
  local errors=0
  local line_num=0

  while IFS= read -r line || [[ -n "$line" ]]; do
    (( ++line_num ))
    [[ -z "${line// /}" ]] && continue
    [[ "$line" =~ ^[[:space:]]*# ]] && continue

    local first_word
    first_word=$(echo "$line" | awk '{print $1}')
    case "$first_word" in
      auto|allow-hotplug|iface|mapping|source|source-directory)
        ;;
      address|netmask|gateway|broadcast|network|metric|mtu|hwaddress|pointopoint)
        ;;
      bridge_ports|bridge_stp|bridge_fd|bridge_maxwait|bridge_waitport)
        ;;
      up|down|pre-up|post-up|pre-down|post-down)
        ;;
      *)
        msg "${YELLOW}Предупреждение [строка $line_num]: Неизвестная или нестандартная
директива '${first_word}'${NOFORMAT}"
        ;;
    esac
  done < "$CONFIG_FILE"

  if [[ $errors -eq 0 ]]; then
    msg "${GREEN}Проверка завершена: синтаксических ошибок не обнаружено.${NOFORMAT}"
  else
    die "Обнаружено $errors ошибок валидации в $CONFIG_FILE" 2
  fi
}

parse_params() {
  while [[ $# -gt 0 ]]; do
```

```
    case "$1" in
      -h|--help)
        usage ;;
      --version)
        version ;;
      -c|--config)
        [[ $# -ge 2 ]] || die "Опция --config требует указания пути к файлу" 2
        CONFIG_FILE="$2"; shift 2 ;;
      -d|--dry-run)
        DRY_RUN=1; shift ;;
      -a|--apply)
        APPLY_NOW=1; shift ;;
      -v|--verbose)
        VERBOSE=1; set -x; shift ;;
      --no-color)
        NO_COLOR_VAL=1; shift ;;
      -*)
        die "Неизвестная опция: '$1'" 2 ;;
      *)
        break ;;
    esac
  done

  [[ $# -gt 0 ]] || usage 2

  local command=$1; shift
  case "$command" in
    show)
      cmd_show "$@" ;;
    set-iface)
      [[ $# -ge 1 ]] || die "Команда set-iface требует указания имени интерфейса" 2
      cmd_set_iface "$@" ;;
    add-route)
      [[ $# -ge 1 ]] || die "Команда add-route требует указания имени интерфейса" 2
      cmd_add_route "$@" ;;
    del-route)
      [[ $# -ge 1 ]] || die "Команда del-route требует указания имени интерфейса" 2
      cmd_del_route "$@" ;;
    create-bridge)
      [[ $# -ge 1 ]] || die "Команда create-bridge требует указания имени моста" 2
      cmd_create_bridge "$@" ;;
    del-iface|del-bridge)
      [[ $# -ge 1 ]] || die "Команда del-iface требует указания имени интерфейса" 2
      remove_stanza "$CONFIG_FILE" "$1" ;;
    up)
      [[ $# -ge 1 ]] || die "Команда up требует указания имени интерфейса" 2
      cmd_up "$1" ;;
    down)
      [[ $# -ge 1 ]] || die "Команда down требует указания имени интерфейса" 2
      cmd_down "$1" ;;
    validate)
      cmd_validate "$@" ;;
    *)
      die "Неизвестная команда: '$command'. Используйте --help для просмотра справки." 2
;;
  esac
}

setup_colors
```

```
parse_params "$@"
```

ПРИЛОЖЕНИЕ Б

Исходный код скрипта установки install.sh

```
#!/usr/bin/env bash
# Скрипт установки утилиты netconfig в системный каталог /usr/local/bin (ЛР №4, вариант
6).
set -Eeuo pipefail

script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" &>/dev/null && pwd -P)
target_bin="/usr/local/bin/netconfig"

echo "=== Установка утилиты netconfig ==="

# Проверка прав суперпользователя
if [[ $EUID -ne 0 ]]; then
  echo "Ошибка: Для установки в /usr/local/bin требуются права суперпользователя (root)."
>&2
  echo "Запустите скрипт через sudo: sudo bash install.sh" >&2
  exit 1
fi

# Проверка обязательных зависимостей
echo "Проверка системных зависимостей..."
for cmd in bash awk ip install mktemp cp chmod rm; do
  if ! command -v "$cmd" >/dev/null 2>&1; then
    echo "Ошибка: Обязательная утилита '$cmd' не найдена в PATH." >&2
    exit 1
  fi
done

source_bin="${script_dir}/netconfig"
if [[ ! -f "$source_bin" ]]; then
  echo "Ошибка: Исходный файл '$source_bin' не найден." >&2
  exit 1
fi

echo "Копирование исполняемого файла в ${target_bin}..."
install -m 0755 "$source_bin" "$target_bin"

echo "Проверка работоспособности установленной утилиты..."
"$target_bin" --version

echo ""
echo "Установка успешно завершена!"
echo "Теперь утилита доступна глобально в системе по команде: netconfig"
echo "Для получения справки выполните: netconfig --help"
```

ПРИЛОЖЕНИЕ В

Исходный код автоматического тестового набора test_netconfig.sh

```
#!/usr/bin/env bash
# Автоматический набор тестов для утилиты netconfig (ЛР №4, вариант 6).
set -Eeuo pipefail

script_dir=$(cd "$(dirname "${BASH_SOURCE[0]}")" &>/dev/null && pwd -P)
netcfg="${script_dir}/../netconfig"

RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
CYAN='\033[0;36m'
NOFORMAT='\033[0m'

test_count=0
fail_count=0

pass() {
  local name=$1
  echo -e "  [${GREEN}PASS${NOFORMAT}] ${name}"
}

fail() {
  local name=$1
  local reason=${2:-""}
  echo -e "  [${RED}FAIL${NOFORMAT}] ${name} - ${reason}"
  (( ++fail_count ))
}

run_test() {
  local name=$1
  shift
  (( ++test_count ))
  if "$@"; then
    pass "$name"
  else
    fail "$name" "Command returned non-zero exit code"
  fi
}

echo -e "${CYAN}=== Запуск тестов для утилиты netconfig ===${NOFORMAT}"

test_dir=$(mktemp -d "/tmp/netconfig-test.XXXXXX")
cleanup() {
  rm -rf "$test_dir"
}
trap cleanup EXIT

test_config="${test_dir}/interfaces"

# 1. Проверка справки и версии
```

```
test_help_version() {
  bash "$netcfg" --help >/dev/null
  bash "$netcfg" --version >/dev/null
}
run_test "Проверка --help и --version" test_help_version


# 2. Проверка обработки ошибок аргументов
test_invalid_args() {
  if bash "$netcfg" 2>/dev/null; then return 1; fi
  if bash "$netcfg" unknown-cmd 2>/dev/null; then return 1; fi
  if bash "$netcfg" --config "$test_config" set-iface eth0 2>/dev/null; then return 1; fi
  if bash "$netcfg" --config "$test_config" set-iface eth0 --ip 999.1.1.1 --netmask
255.255.255.0 2>/dev/null; then return 1; fi
  if bash "$netcfg" --config "$test_config" set-iface eth0 --ip 192.168.1.10 --netmask
255.255.255.123 2>/dev/null; then return 1; fi
  return 0
}
run_test "Проверка обработки некорректных аргументов и IP" test_invalid_args


# 3. Настройка статического сетевого интерфейса (IP, маска, шлюз)
test_set_iface_static() {
  bash "$netcfg" --config "$test_config" set-iface eth0 \
    --ip 192.168.10.50 \
    --netmask 255.255.255.0 \
    --gateway 192.168.10.1


  grep -q "auto eth0" "$test_config" || return 1
  grep -q "iface eth0 inet static" "$test_config" || return 1
  grep -q "address 192.168.10.50" "$test_config" || return 1
  grep -q "netmask 255.255.255.0" "$test_config" || return 1
  grep -q "gateway 192.168.10.1" "$test_config" || return 1
  return 0
}
run_test "Настройка статического интерфейса eth0 (IP, маска, шлюз)" test_set_iface_static


# 4. Настройка интерфейса через нотацию CIDR
test_set_iface_cidr() {
  bash "$netcfg" --config "$test_config" set-iface eth1 \
    --ip 10.0.5.100/24


  grep -q "auto eth1" "$test_config" || return 1
  grep -q "iface eth1 inet static" "$test_config" || return 1
  grep -q "address 10.0.5.100" "$test_config" || return 1
  grep -q "netmask 255.255.255.0" "$test_config" || return 1
  return 0
}
run_test "Настройка интерфейса eth1 через CIDR (10.0.5.100/24)" test_set_iface_cidr


# 5. Идемпотентность обновления конфигурации интерфейса
test_iface_update_idempotent() {
  bash "$netcfg" --config "$test_config" set-iface eth0 \
    --ip 192.168.10.60 \
    --netmask 255.255.255.0 \
    --gateway 192.168.10.254


  local count
  count=$(grep -c "iface eth0 inet static" "$test_config")
```

```
  [[ $count -eq 1 ]] || return 1
  grep -q "address 192.168.10.60" "$test_config" || return 1
  grep -q "gateway 192.168.10.254" "$test_config" || return 1
  return 0
}
run_test "Идемпотентное обновление настроек интерфейса eth0" test_iface_update_idempotent


# 6. Добавление персистентного статического маршрута
test_add_route() {
  bash "$netcfg" --config "$test_config" add-route eth0 \
    --network 172.16.0.0/16 \
    --via 192.168.10.254


  grep -q "up ip route add 172.16.0.0/16 via 192.168.10.254 dev eth0 || true"
"$test_config" || return 1
  grep -q "down ip route del 172.16.0.0/16 via 192.168.10.254 dev eth0 || true"
"$test_config" || return 1
  return 0
}
run_test "Добавление персистентного маршрута (172.16.0.0/16 via 192.168.10.254)"
test_add_route


# 7. Проверка исключения дублирования маршрута
test_route_no_duplicate() {
  bash "$netcfg" --config "$test_config" add-route eth0 \
    --network 172.16.0.0/16 \
    --via 192.168.10.254


  local count
  count=$(grep -c "up ip route add 172.16.0.0/16 via 192.168.10.254" "$test_config")
  [[ $count -eq 1 ]] || return 1
  return 0
}
run_test "Защита от дублирования идентичного маршрута" test_route_no_duplicate


# 8. Создание сетевого моста (bridge) с привязкой интерфейсов
test_create_bridge() {
  bash "$netcfg" --config "$test_config" create-bridge br0 \
    --ports eth1,eth2 \
    --ip 192.168.100.1/24 \
    --stp off \
    --fd 0


  grep -q "auto br0" "$test_config" || return 1
  grep -q "iface br0 inet static" "$test_config" || return 1
  grep -q "address 192.168.100.1" "$test_config" || return 1
  grep -q "bridge_ports eth1 eth2" "$test_config" || return 1
  grep -q "bridge_stp off" "$test_config" || return 1
  grep -q "bridge_fd 0" "$test_config" || return 1
  return 0
}
run_test "Создание сетевого моста br0 с подчиненными портами eth1 eth2"
test_create_bridge


# 9. Удаление статического маршрута
test_del_route() {
  bash "$netcfg" --config "$test_config" del-route eth0 \
```

```
    --network 172.16.0.0/16

  if grep -q "172.16.0.0/16" "$test_config"; then
    return 1
  fi
  return 0
}
run_test "Удаление статического маршрута из конфигурации eth0" test_del_route

# 10. Удаление интерфейса / моста
test_del_iface() {
  bash "$netcfg" --config "$test_config" del-iface eth1
  if grep -q "iface eth1" "$test_config"; then
    return 1
  fi
  return 0
}
run_test "Удаление интерфейса eth1 из файла конфигурации" test_del_iface

# 11. Проверка синтаксиса (validate)
test_validate() {
  bash "$netcfg" --config "$test_config" validate >/dev/null
}
run_test "Валидация синтаксиса и структуры конфигурации (validate)" test_validate

# 12. Проверка отображения (show)
test_show() {
  local out
  out=$(bash "$netcfg" --config "$test_config" show)
  echo "$out" | grep -q "eth0" || return 1
  echo "$out" | grep -q "br0" || return 1
  return 0
}
run_test "Отображение состояния интерфейсов и мостов (show)" test_show

# 13. Проверка режима dry-run (без изменения файла)
test_dry_run() {
  local mtime_before
  mtime_before=$(stat -c %Y "$test_config")
  sleep 1
  bash "$netcfg" --config "$test_config" --dry-run set-iface eth99 --ip 1.2.3.4/24
>/dev/null
  local mtime_after
  mtime_after=$(stat -c %Y "$test_config")
  [[ "$mtime_before" == "$mtime_after" ]] || return 1
  if grep -q "eth99" "$test_config"; then return 1; fi
  return 0
}
run_test "Проверка режима предпросмотра (--dry-run)" test_dry_run

# 14. Проверка включения и отключения в режиме dry-run (up / down)
test_up_down_dry_run() {
  bash "$netcfg" --dry-run up br0 >/dev/null
  bash "$netcfg" --dry-run down br0 >/dev/null
}
run_test "Проверка команд up и down (--dry-run)" test_up_down_dry_run
```

```
# 15. Проверка автоматического создания резервной копии (.bak)
test_backup_created() {
  local bak_count
  bak_count=$(ls -1 "${test_config}".bak.* 2>/dev/null | wc -l)
  (( bak_count >= 1 )) || return 1
  return 0
}
run_test "Создание резервных копий конфигурационного файла (*.bak.*)" test_backup_created


echo "------------------------------------------------------------"
echo -e "Итог тестов: ${CYAN}${test_count}${NOFORMAT} выполнено, ${GREEN}$(( test_count -
fail_count ))${NOFORMAT} успешно, ${RED}${fail_count}${NOFORMAT} ошибок."


if (( fail_count > 0 )); then
  exit 1
fi
exit 0
```